

Лабораторный практикум 16. Фролов А.А.

Добавленные переменные

Для реализации файлового режима были добавлены переменные:

```
1 ; -----
2 ; Данные для работы с файлом
3 ; -----
4
5 Prf      db 0      ; Признак работы с файлом: 0 - память, 1 - файл
6 Lognum   dw 0      ; Логический номер открытого файла
7 Lf       dw 0      ; Длина файла в секторах по 256 байт
8 Lsec     dw 0      ; Фактическая длина текущего сектора файла
9
10 Msg_file_question db 'Edit file? y/n: $'
11 Msg_file_name     db 0Dh,0Ah,'File name: $'
12 Msg_file_error    db 0Dh,0Ah,'File open error',0Dh,0Ah,'$'
13 Msg_file_ok       db 0Dh,0Ah,'File opened',0Dh,0Ah,'$'
14
15 Msg_n_too_big     db 0Dh,0Ah,'N is too big. Last sector
16 loaded.',0Dh,0Ah,'$'
17 Msg_read_error    db 0Dh,0Ah,'File read error',0Dh,0Ah,'$'
18 Msg_write_error   db 0Dh,0Ah,'File write error',0Dh,0Ah,'$'
19 Msg_no_file_cmd   db 0Dh,0Ah,'Command is not available in file
20 mode',0Dh,0Ah,'$'
21
22 ; Буфер для ввода имени файла через int 21h / ah=0Ah
23 FileNameMax db 80
24 FileNameLen db 0
25 FileName    db 81 dup (0)
```

Переменная `Prf` определяет текущий режим работы редактора. Если `Prf = 0`, программа работает с памятью. Если `Prf = 1`, программа работает с файлом.

Переменная `Lognum` хранит логический номер открытого файла, который возвращается функцией `int 21h / 3Dh`.

Переменная `Lf` содержит длину файла в секторах по 256 байт, а `Lsec` хранит фактическое количество байт, считанных из текущего сектора файла. Это важно для последнего сектора, который может быть меньше 256 байт.

Подключение начальной фазы диспетчера

В основной диспетчер программы был добавлен вызов процедуры `Init_file_mode`.

```

1  Dispatcher:
2      call Clear_screen
3
4      ; Начальная фаза диспетчера для лабораторной работы 16.
5      ; Здесь программа спрашивает, нужно ли редактировать файл.
6      ; Если выбран файл, он открывается и определяется его длина.
7      call Init_file_mode
8
9  Dispatcher_loop:
10     call Send_crlf
11
12     mov dl,'>'
13     call write_char
14
15     call Read_byte          ; AL = ASCII-код, AH = скан-код
16
17     cmp ah,44h              ; F10?
18     je Dispatcher_exit
19
20     cmp al,00h              ; управляющая клавиша?
21     jne Dispatcher_loop
22
23     mov dl,ah               ; DL = скан-код команды
24     call Command
25
26     jmp Dispatcher_loop

```

Процедура `Init_file_mode` вызывается один раз при запуске редактора. Она определяет, будет ли программа работать с памятью или с файлом. После этого управление передаётся прежнему циклу диспетчера, который обрабатывает функциональные клавиши.

Процедура выбора файлового режима

```

1  ; -----
2  ; Init_file_mode
3  ; Начальная фаза диспетчера для лабораторной работы 16
4  ;
5  ; Спрашивает пользователя, нужно ли редактировать файл.
6  ; Если пользователь вводит Y/y:
7  ;   - запрашивает имя файла;
8  ;   - открывает файл на чтение и запись;
9  ;   - определяет длину файла в секторах по 256 байт;
10 ;   - устанавливает Prf = 1.
11 ; Если пользователь вводит другой символ:
12 ;   - устанавливает Prf = 0, редактор работает с памятью.
13 ; -----
14

```

```

15 Init_file_mode:
16     push ax
17     push bx
18     push cx
19     push dx
20
21     ; По умолчанию работаем с памятью
22     mov byte ptr [Prf],0
23
24     ; Вопрос о работе с файлом
25     mov dx,offset Msg_file_question
26     mov ah,09h
27     int 21h
28
29     ; Ввод одного символа
30     mov ah,01h
31     int 21h
32
33     ; Перевод Y в y
34     or al,20h
35     cmp al,'y'
36     jne Init_file_mode_exit
37
38     ; Запрос имени файла
39     mov dx,offset Msg_file_name
40     mov ah,09h
41     int 21h
42
43     ; Ввод имени файла
44     mov dx,offset FileNameMax
45     mov ah,0Ah
46     int 21h
47
48     ; DOS 0Ah оставляет в конце строки код Enter 0Dh.
49     ; Для функций работы с файлами строка должна заканчиваться нулевым
байтом.
50     xor bx,bx
51     mov bl,[FileNameLen]
52     mov byte ptr [FileName+bx],0
53
54     ; Открытие файла на чтение и запись
55     mov dx,offset FileName
56     mov al,02h                ; 02h - открыть для чтения и записи
57     mov ah,3Dh
58     int 21h
59     jc Init_file_mode_error
60
61     ; Сохраняем логический номер файла
62     mov [Lognum],ax
63

```

```

64      ; Определение длины файла в байтах:
65      ; установка указателя файла на конец
66      mov bx,[Lognum]
67      xor cx,cx
68      xor dx,dx
69      mov al,02h          ; относительно конца файла
70      mov ah,42h
71      int 21h
72      jc Init_file_mode_error
73
74      ; В CX:DX находится длина файла в байтах.
75      ; Нужно получить количество секторов по 256 байт:
76      ; Lf = (длина + 255) / 256
77      mov bx,dx
78      xor dx,dx
79      mov ax,255
80      add ax,bx
81      adc dx,cx
82      mov bx,256
83      div bx
84      mov [Lf],ax
85
86      ; Если Lf > 256, ограничиваем редактируемую часть файла
87      cmp ax,256
88      jbe Init_file_mode_set_flag
89
90      mov word ptr [Lf],256
91
92      Init_file_mode_set_flag:
93      mov byte ptr [Prf],1
94
95      ; Возвращаем указатель файла в начало
96      mov bx,[Lognum]
97      xor cx,cx
98      xor dx,dx
99      mov al,00h
100     mov ah,42h
101     int 21h
102
103     ; Сообщение об успешном открытии
104     mov dx,offset Msg_file_ok
105     mov ah,09h
106     int 21h
107
108     jmp Init_file_mode_exit
109
110     Init_file_mode_error:
111     mov dx,offset Msg_file_error
112     mov ah,09h
113     int 21h

```

```

114     mov byte ptr [Prf],0
115
116 Init_file_mode_exit:
117     pop dx
118     pop cx
119     pop bx
120     pop ax
121     ret

```

Данная процедура реализует начальную фазу нового диспетчера. Сначала программа спрашивает пользователя, нужно ли редактировать файл. Если пользователь вводит `y` или `Y`, программа запрашивает имя файла.

Имя файла вводится функцией `int 21h / 0Ah`. После ввода в конец строки записывается нулевой байт, так как функции DOS для работы с файлами требуют строку, завершающуюся нулём.

Затем файл открывается функцией `int 21h / 3Dh` в режиме чтения и записи. Если открытие прошло успешно, логический номер файла сохраняется в переменной `Lognum`.

Для определения длины файла указатель файла устанавливается в конец с помощью функции `int 21h / 42h`. Полученная длина в байтах переводится в количество секторов по 256 байт. Если размер файла больше 256 секторов, значение ограничивается числом 256.

Изменённая процедура `N_sector`

```

1  ; -----
2  ; Загрузка N-го сектора
3  ; Если Prf = 0, сектор берется из памяти.
4  ; Если Prf = 1, сектор читается из открытого файла.
5  ; N вводится как двухзначное шестнадцатеричное число.
6  ; -----
7  N_sector:
8      push ax
9      push bx
10     push cx
11     push dx
12     push si
13     push di
14
15     call Send_crlf
16
17     mov dl,'N'
18     call write_char
19     mov dl,'='
20     call write_char

```

```

21
22     call Read_byte_hex      ; AL = номер сектора
23
24     ; Address = N00h
25     mov ah,al
26     mov al,0
27     mov [Address],ax
28
29     ; Проверка режима работы
30     cmp byte ptr [Prf],0
31     jne N_sector_file_mode
32
33     ; ----- старый режим: работа с памятью -----
34 N_sector_memory_mode:
35     call Send_crlf
36
37     mov si,[Address]
38     mov di,offset Sector
39     call Copy_sector
40
41     call disp_sector
42
43     jmp N_sector_exit
44
45
46     ; ----- новый режим: работа с файлом -----
47 N_sector_file_mode:
48     ; Проверяем номер сектора.
49     ; Если Lf = 256, то допустимы все номера 00-FF.
50     mov bx,[Lf]
51     cmp bx,256
52     je N_sector_file_ok
53
54     ; Если файл пустой, оставляем сектор 0
55     cmp bx,0
56     jne N_sector_check_size
57
58     mov word ptr [Address],0
59     jmp N_sector_file_ok
60
61 N_sector_check_size:
62     ; BX = Lf * 256
63     mov bh,bl
64     mov bl,0
65
66     mov ax,[Address]
67     cmp ax,bx
68     jb N_sector_file_ok
69
70     ; Если N слишком большое, загружаем последний сектор файла

```

```

71     mov dx,offset Msg_n_too_big
72     mov ah,09h
73     int 21h
74
75     mov ax,[Lf]
76     dec ax
77     mov ah,al
78     mov al,0
79     mov [Address],ax
80
81 N_sector_file_ok:
82     ; Установка указателя файла на Address
83     mov bx,[Lognum]
84     xor cx,cx
85     mov dx,[Address]
86     mov al,00h                ; перемещение относительно начала файла
87     mov ah,42h
88     int 21h
89     jc N_sector_read_error
90
91     ; Обнуление Sector перед чтением.
92     ; Это нужно, если последний сектор файла меньше 256 байт.
93     push es
94     push ds
95     pop es
96     mov di,offset Sector
97     xor ax,ax
98     mov cx,128
99     cld
100    rep stosw
101    pop es
102
103    ; Чтение сектора файла
104    mov bx,[Lognum]
105    mov cx,256
106    mov dx,offset Sector
107    mov ah,3Fh
108    int 21h
109    jc N_sector_read_error
110
111    ; AX = фактически считанное количество байт
112    mov [Lsec],ax
113
114    call disp_sector
115
116    jmp N_sector_exit
117
118 N_sector_read_error:
119     mov dx,offset Msg_read_error
120     mov ah,09h

```

```

121     int 21h
122
123     N_sector_exit:
124         pop di
125         pop si
126         pop dx
127         pop cx
128         pop bx
129         pop ax
130         ret

```

Процедура `N_sector` была изменена так, чтобы она могла работать в двух режимах.

Если `Prf = 0`, выполняется прежний алгоритм: по введённому номеру сектора вычисляется адрес `N00h`, после чего 256 байт копируются из памяти в буфер `Sector`.

Если `Prf = 1`, процедура работает с файлом. Сначала проверяется, не превышает ли введённый номер сектора фактическую длину файла. Если номер слишком большой, программа выводит сообщение и загружает последний доступный сектор.

После этого указатель файла устанавливается на позицию `Address` с помощью функции `int 21h / 42h`. Затем буфер `Sector` очищается и в него считывается до 256 байт из файла с помощью функции `int 21h / 3Fh`.

Количество реально считанных байт сохраняется в переменной `Lsec`.

Изменённая процедура `Write_sector`

```

1  ; -----
2  ; Запись содержимого Sector
3  ; Если Prf = 0, Sector записывается в память.
4  ; Если Prf = 1, Sector записывается в открытый файл.
5  ; -----
6  Write_sector:
7      push ax
8      push bx
9      push cx
10     push dx
11     push si
12     push di
13
14     ; Проверка режима работы
15     cmp byte ptr [Prf],0
16     jne Write_sector_file_mode
17

```



```

18 ; ----- старый режим: запись в память -----
19 Write_sector_memory_mode:
20     mov si,offset Sector
21     mov di,[Address]
22     call Copy_sector
23
24     jmp Write_sector_exit
25
26
27 ; ----- новый режим: запись в файл -----
28 Write_sector_file_mode:
29     ; Установка указателя файла на позицию Address
30     mov bx,[Lognum]
31     xor cx,cx
32     mov dx,[Address]
33     mov al,00h           ; от начала файла
34     mov ah,42h
35     int 21h
36     jc Write_sector_error
37
38     ; Запись содержимого Sector в файл
39     mov bx,[Lognum]
40     mov cx,[Lsec]         ; записываем фактическую длину сектора
41     mov dx,offset Sector
42     mov ah,40h
43     int 21h
44     jc Write_sector_error
45
46     jmp Write_sector_exit
47
48 Write_sector_error:
49     mov dx,offset Msg_write_error
50     mov ah,09h
51     int 21h
52
53 Write_sector_exit:
54     pop di
55     pop si
56     pop dx
57     pop cx
58     pop bx
59     pop ax
60     ret

```

Процедура `Write_sector` также была разделена на два режима.

В режиме работы с памятью она выполняет прежнее действие: копирует содержимое буфера `Sector` обратно в область памяти по адресу `Address`.

В файловом режиме процедура устанавливает указатель файла на позицию `Address` , а затем записывает содержимое буфера `Sector` в файл. Для записи используется функция `int 21h / 40h` .

Количество записываемых байт берётся из переменной `Lsec` . Это позволяет корректно записывать последний сектор файла, если он содержит меньше 256 байт.

Ограничение процедур `Init_sector` , `Prev_sector` , `Next_sector`

В файловом режиме процедуры `Init_sector` , `Prev_sector` и `Next_sector` не должны выполнять прежний алгоритм, так как они предназначены для работы с памятью. Поэтому в начало каждой процедуры была добавлена проверка переменной `Prf` .

Процедура `Init_sector`

```
1  ; -----
2  ; Инициализация сектора
3  ; Если Prf = 0, работает прежний алгоритм.
4  ; Если Prf = 1, команда запрещена в файловом режиме.
5  ; -----
6  Init_sector:
7      cmp byte ptr [Prf],0
8      jne Init_sector_file_mode
9
10     push si
11     push di
12
13     mov word ptr [Address],0
14
15     mov si,0
16     mov di,offset Sector
17     call Copy_sector
18
19     call disp_sector
20
21     pop di
22     pop si
23     ret
24
25 Init_sector_file_mode:
26     mov dx,offset Msg_no_file_cmd
27     mov ah,09h
28     int 21h
29     ret
```

Процедура Prev_sector

```
1 ; -----
2 ; Загрузка предыдущего сектора памяти
3 ; Если Prf = 0, работает прежний алгоритм.
4 ; Если Prf = 1, команда запрещена в файловом режиме.
5 ; -----
6 Prev_sector:
7     cmp byte ptr [Prf],0
8     jne Prev_sector_file_mode
9
10    push si
11    push di
12
13    mov ax,[Address]
14
15    cmp ax,0
16    je Prev_no_change
17
18    sub ax,100h
19    mov [Address],ax
20
21 Prev_no_change:
22    mov si,[Address]
23    mov di,offset Sector
24    call Copy_sector
25
26    call disp_sector
27
28    pop di
29    pop si
30    ret
31
32 Prev_sector_file_mode:
33    mov dx,offset Msg_no_file_cmd
34    mov ah,09h
35    int 21h
36    ret
```

Процедура Next_sector

```
1 ; -----
2 ; Загрузка следующего сектора памяти
3 ; Если Prf = 0, работает прежний алгоритм.
4 ; Если Prf = 1, команда запрещена в файловом режиме.
5 ; -----
6 Next_sector:
7     cmp byte ptr [Prf],0
8     jne Next_sector_file_mode
```

```

9
10     push si
11     push di
12
13     mov ax,[Address]
14
15     cmp ax,0FF00h
16     je Next_no_change
17
18     add ax,100h
19     mov [Address],ax
20
21 Next_no_change:
22     mov si,[Address]
23     mov di,offset Sector
24     call Copy_sector
25
26     call disp_sector
27
28     pop di
29     pop si
30     ret
31
32 Next_sector_file_mode:
33     mov dx,offset Msg_no_file_cmd
34     mov ah,09h
35     int 21h
36     ret

```

Данные процедуры оставлены для работы с памятью. При файловом режиме они выводят сообщение о недоступности команды и завершаются без изменения данных. Это соответствует ограничению лабораторной работы: для работы с файлами были модернизированы только процедуры `N_sector` и `Write_sector`.

Описание работы программы

После запуска программа очищает экран и задаёт пользователю вопрос:

```
Edit file? y/n:
```

Если пользователь вводит `n`, редактор работает в прежнем режиме с областью оперативной памяти.

Если пользователь вводит `y`, программа запрашивает имя файла:

```
File name:
```

После ввода имени существующего файла программа открывает его на чтение и запись, определяет длину в секторах и переходит к основному режиму обработки

команд.

В файловом режиме используются следующие клавиши:

Клавиша	Действие
F6	Ввод номера сектора и чтение сектора из файла
F4	Переход в режим редактирования сектора
F2	Запись изменённого сектора обратно в файл
F10	Завершение программы

Клавиши F1 , F3 , F5 в файловом режиме не используются и выводят сообщение о недоступности команды.

План проверки программы

Для проверки работы программы необходимо создать тестовый файл, например TEST.TXT , содержащий простой текст:

```
ABCDEFGG 1234567890 test file
```

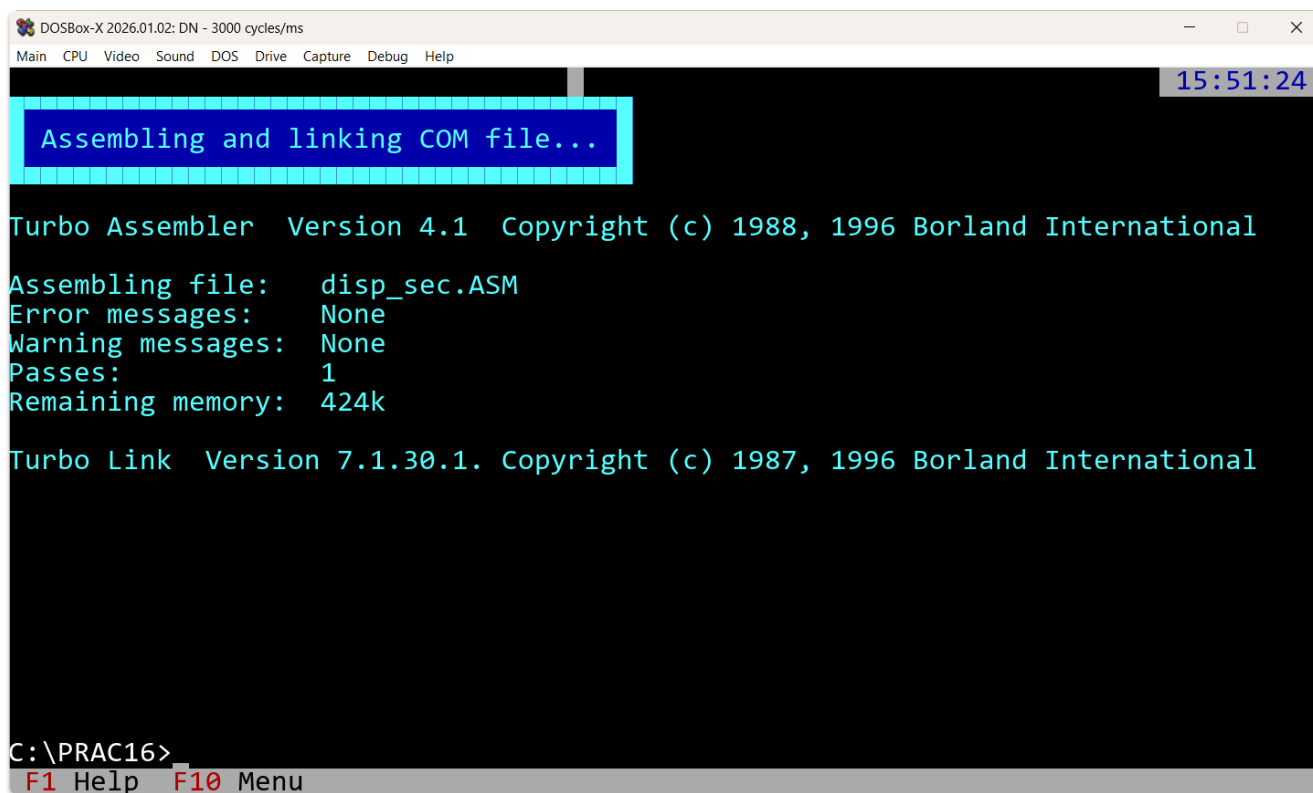
Далее программа запускается в DOSBox-X. При запуске выбирается файловый режим, вводится имя файла TEST.TXT , затем с помощью клавиши F6 загружается сектор 00 .

После вывода содержимого сектора на экран выполняется редактирование одного или нескольких байтов с помощью клавиши F4 . После выхода из режима редактирования изменённый сектор записывается обратно в файл клавишей F2 .

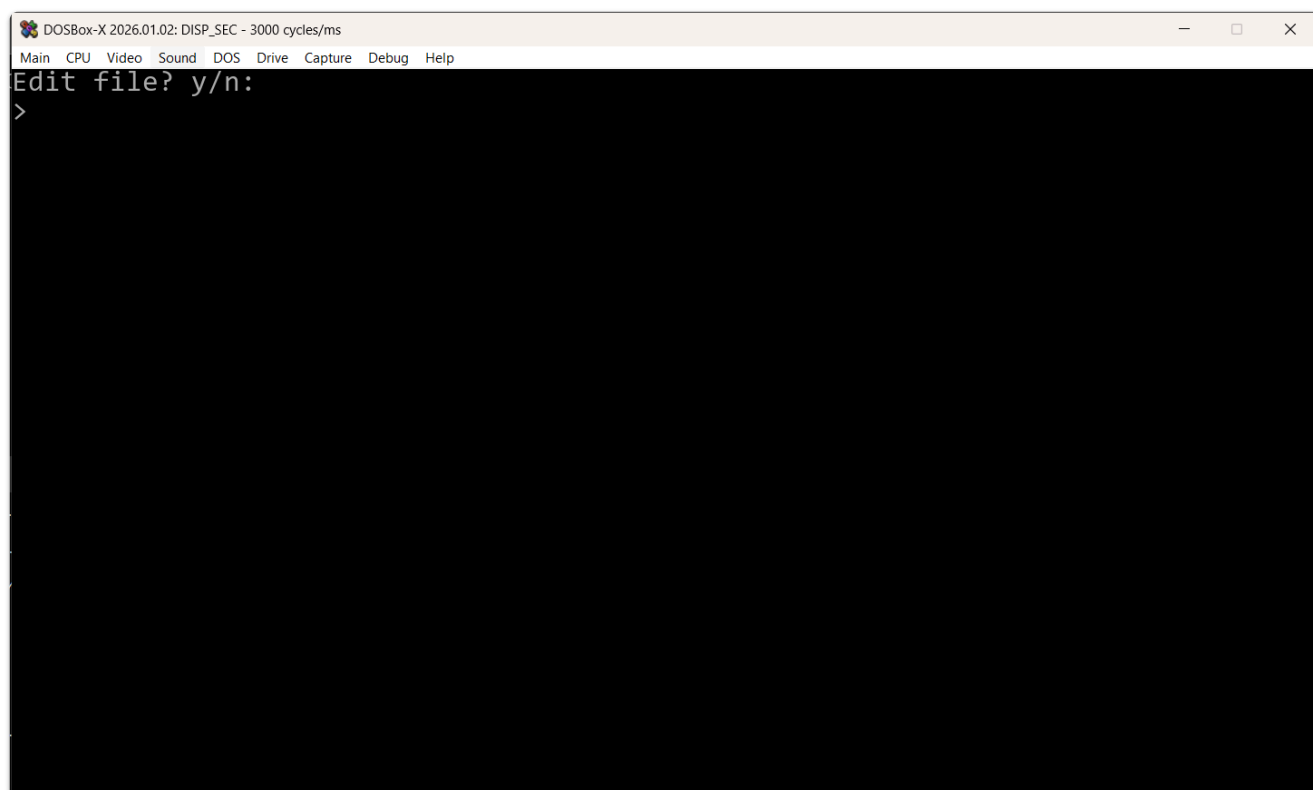
Для подтверждения результата файл можно открыть повторно или вывести его содержимое командой DOS, чтобы убедиться, что изменения были сохранены.

Результаты проверки

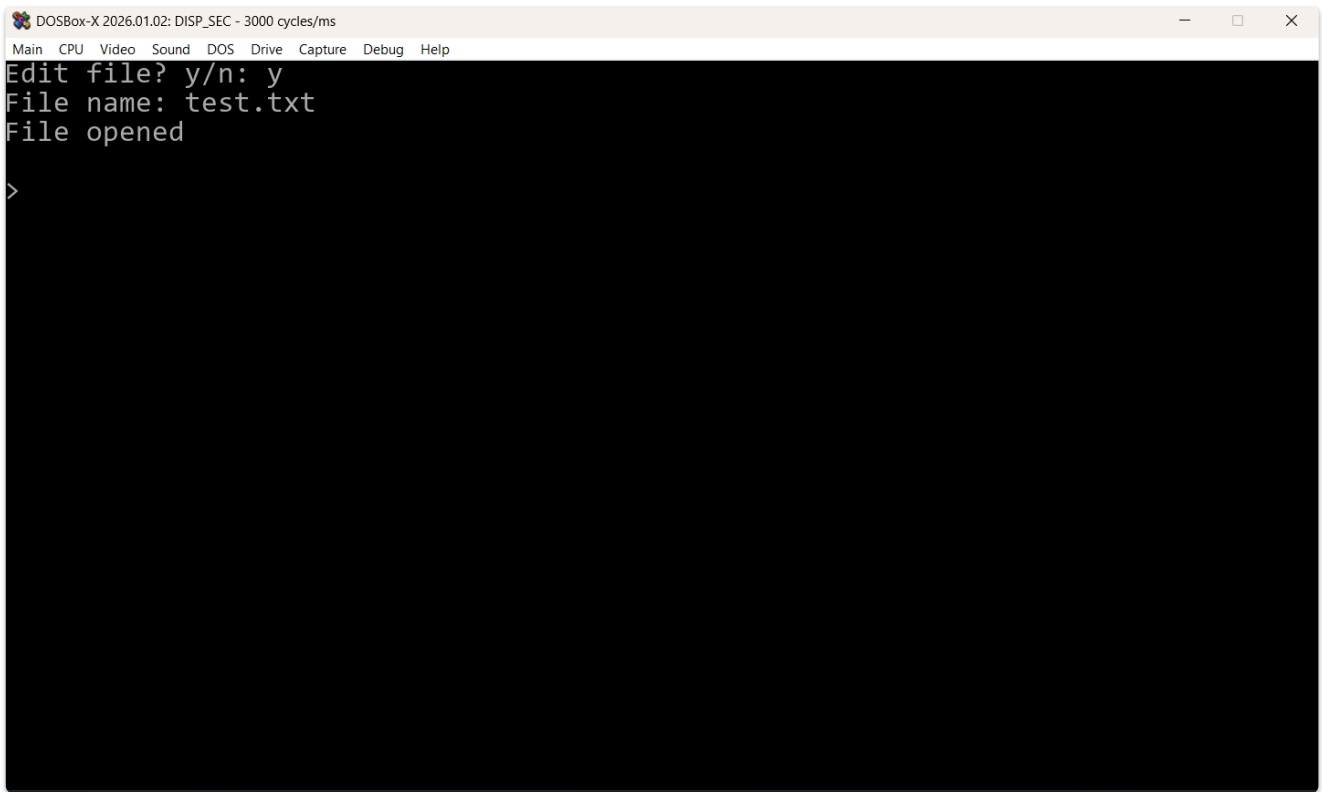
[Скриншот 1 — Компиляция программы без ошибок](#)



Скриншот 2 — Запуск программы и выбор файлового режима



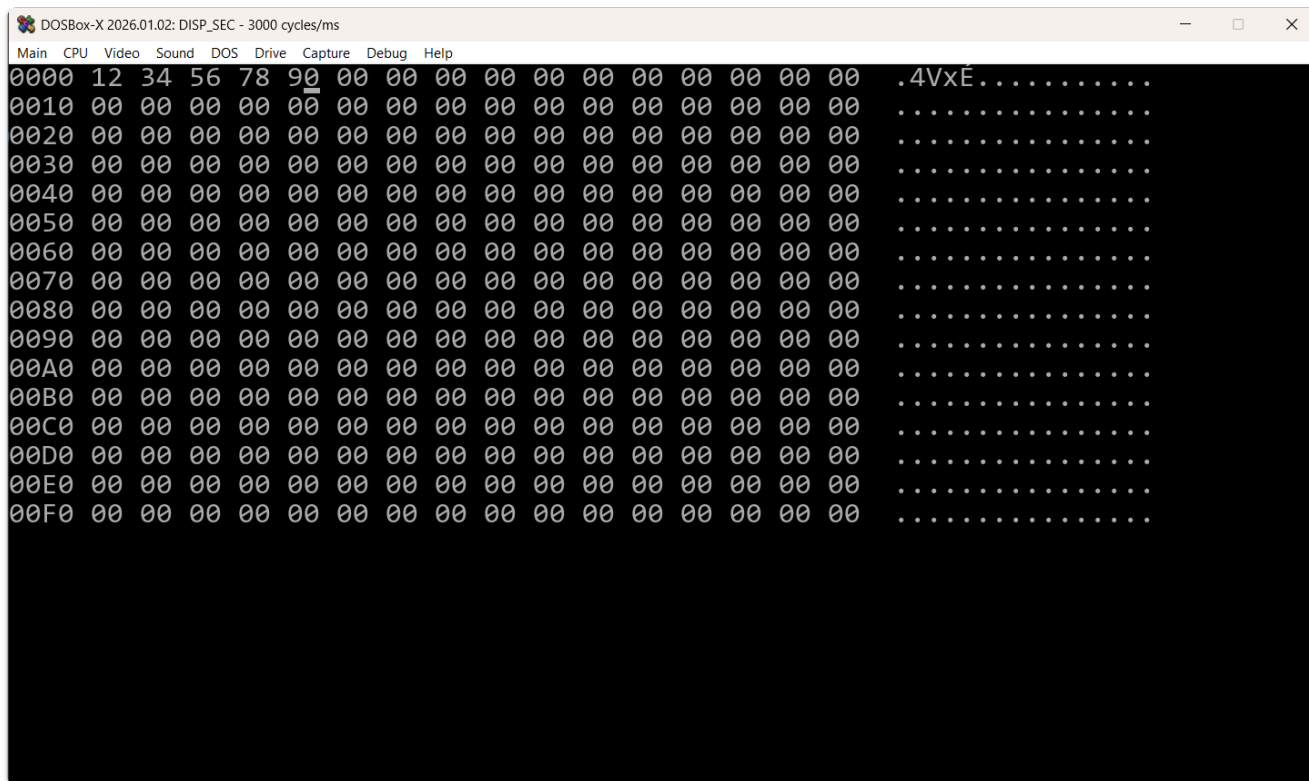
Скриншот 3 — Открытие тестового файла



Скриншот 4 — Загрузка сектора файла



Скриншот 5 — Редактирование сектора



Вывод

В ходе лабораторной работы была выполнена модернизация редактора информации. В программу был добавлен файловый режим работы, позволяющий открывать существующий файл, определять его длину, считывать выбранный сектор файла в буфер, редактировать его и записывать изменения обратно в файл.

Были изучены и применены системные функции DOS для работы с файлами: открытие файла, установка указателя файла, чтение и запись. Программа сохранила прежний режим работы с оперативной памятью, но получила дополнительную возможность редактирования содержимого файла.